



University of
Nottingham

UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Attendance Monitoring
QR Code 

Lecture 1 – Introduction and Basics

Prof Ender Özcan

Dr Warren G Jackson

**What do the
following
things have
in common?**





Timetables

■ Nurse Rostering

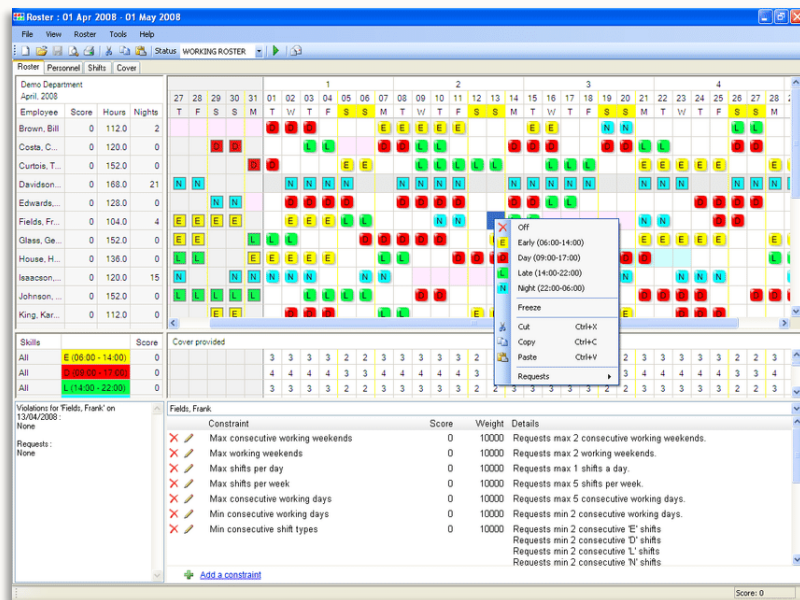


Image source: Burke, Edmund & Curtois, Timothy & Qu, Rong & Vanden Berghe, Greet. (2010). A scatter search methodology for the nurse rostering problem. JORS. 61. 1667-1679. 10.1057/jors.2009.118.

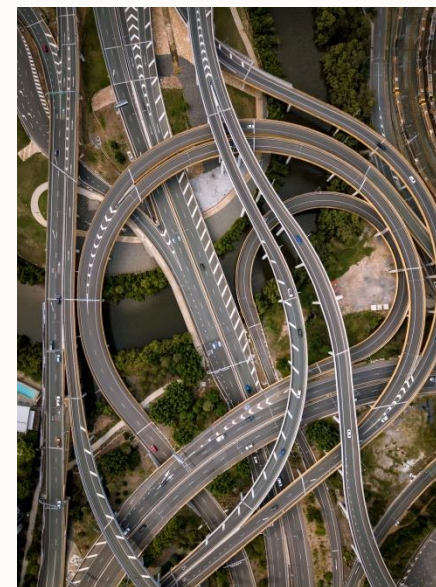
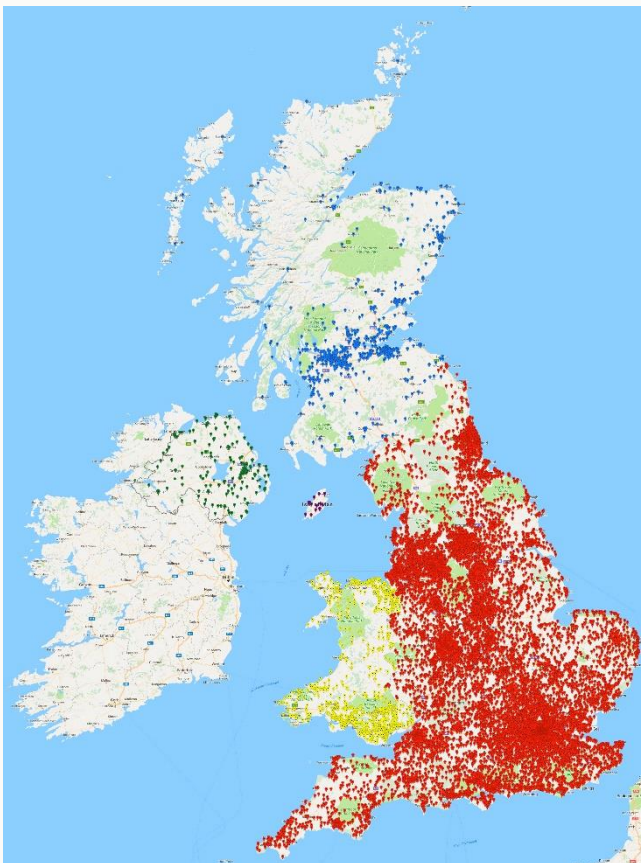
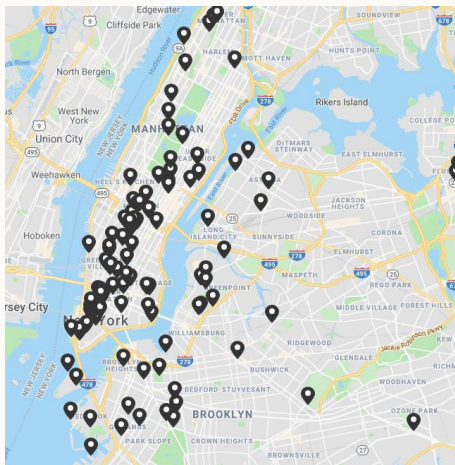
Programme: BSc Hons Comp Sci with AI/F/02																Weeks: [19-37] w19 - w/c Mon 26/01/26-w37 - w/c Mon 01/06/26				
G4G7 Computer Science with Artificial Intelligence																				
	8:30	9:00	9:30	10:00	10:30	11:00	11:30	12:00	12:30	13:00	13:30	14:00	14:30	15:00	15:30	16:00	16:30	17:00	17:30	
Mon				COMP/2012/01/L/01/01 Lecture Languages and Computation Widdershoven, Cas Dr, Alenkirch, Thorsten Prof 19-27, 32, 34 JC Business School South Auditorium B52		COMP/2003/01/L/01/01 Lecture Advanced Functional Programming Hutton, Graham Prof 19-27, 32, 34 JC Exchange LT1				COMP/2004/02/L/01/01 Lecture Algorithms Data Structures and Efficiency Parkes, Andrew Dr 19-27, 32, 34 JC Business School South Auditorium B52		COMP/2002/2053/01/C/02/01 Software Engineering Group Marsden, Dan Dr, De Maere, Geert Dr 19-27, 32, 34 JC Computer Science Atrium, JC Computer Science A07, JC Computer Science A32			COMP/2002/2053/01/H/02/01 Hybrid Delivery Marsden, Dan Dr, De Maere, Geert Dr 19, 27 JC Business School South A25		COMP/2004/02/T/01/04 Tutorial Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32, 34 JC Advanced Manufacturing C011			
			COMP/2014/01/L/01/01 Lecture Distributed Systems Greenhalgh, Chris Prof 19-27, 32-33 JC Business School South A25			COMP/2004/02/C/01/01 Computing Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Computer Science Atrium, JC Computer Science A32		COMP/2004/02/C/01/02 Computing Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Computer Science Atrium, JC Computer Science A32										COMP/2004/02/T/01/02 Tutorial Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Advanced Manufacturing C011		
Tue			COMP/2012/01/T/01/02 Tutorial Languages and Computation Widdershoven, Cas Dr 20-27, 32-34 Manufacturing C011																	
			COMP/2012/01/T/01/01 Tutorial Languages and Computation Alenkirch, Thorsten Prof 20-27, 32, 34 JC Exchange C33	COMP/2010/01/L/01/01 Lecture Software Specification Greensmith, Julie Dr, Luckcruck, M Dr 19-27, 32, 33 JC Exchange C33						COMP/2003/01/C/01/01 Advanced Functional Programming Hutton, Graham Prof 19-27, 32		Computing Hutton, Graham Prof JC Dearing A34	COMP/2006/01/L/01/01 Lecture C++ Programming Chen, Chao Dr 19-27, 32, 33 JC Exchange LT1			COMP/2001/2011/01/L/01/01 Artificial Intelligence Methods Ozcan, Ender Prof, Jackson, Warren Dr 19-27, 32, 33		Lecture Ozcan, Ender Prof, Jackson, Warren Dr JC Exchange LT3		
Wed						COMP/2004/01/L/01/01 Introduction to Cyber Physical Systems Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr 19-27, 32-33														
			COMP/2005/01/L/01/01 Introduction to Image Processing Moemeni, Armaghan Dr 19-27, 32-33		COMP/2004/02/T/01/03 Tutorial Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Exchange C33															
			COMP/2004/01/C/01/01 Introduction to Cyber Physical Systems Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr 19-27, 32-33											COMP/2004/02/T/01/06 Tutorial Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Advanced Manufacturing C011		COMP/2004/01/C/01/02 Introduction to Cyber Physical Systems Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr 19-27, 32-33		Computing Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr JC Computer Science B85		
Thu																				
			COMP/2012/01/T/01/03 Tutorial Languages and Computation Alenkirch, Thorsten Prof, Widdershoven, Cas Dr 20-27, 32-34 JC Advanced Manufacturing C011	COMP/2012/01/T/01/04 Tutorial Languages and Computation Widdershoven, Cas Dr, Alenkirch, Thorsten Prof 20-27, 32-34 JC Exchange B01		COMP/2012/01/T/01/05 Tutorial Languages and Computation Alenkirch, Thorsten Prof, Widdershoven, Cas Dr 20-27, 32-34 JC Exchange B01	COMP/2012/01/T/01/07 Tutorial Languages and Computation Alenkirch, Thorsten Prof, Widdershoven, Cas Dr 20-27, 32-34 JC Exchange B01			COMP/2004/02/L/02/01 Lecture Algorithms Data Structures and Efficiency Parkes, Andrew Dr 19-27, 32-33 JC Business School South Auditorium B52	COMP/2012/01/T/01/06 Tutorial Languages and Computation Alenkirch, Thorsten Prof, Widdershoven, Cas Dr 20-27, 32-34 JC Exchange B01		COMP/2014/01/C/01/01 Computing Algorithms Data Structures and Efficiency Greenhalgh, Chris Prof 19-27, 32, 33 JC Dearing A34		COMP/2006/01/T/01/01 Tutorial C++ Programming Chen, Chao Dr 19-27, 32, 33 JC Exchange C33					
			COMP/2005/01/C/01/01 Introduction to Image Processing Moemeni, Armaghan Dr 19-27, 32-33	Computing Moemeni, Armaghan Dr JC Computer Science A32		COMP/2006/01/L/02/01 Lecture C++ Programming Chen, Chao Dr 19-27, 32, 33 JC Exchange LT1				COMP/2006/01/C/01/01 Computing C++ Programming Chen, Chao Dr 19-27, 32-33 JC Computer Science A32			COMP/2004/01/L/01/01 Lecture Introduction to Cyber Physical Systems Sailaja, Neelima Dr, Deshpande, Nikhila Dr 19-27, 32-33 JC Exchange LT2			COMP/2014/01/C/01/02 Computing Algorithms Data Structures and Efficiency Greenhalgh, Chris Prof 19-27, 32, 33 JC Dearing A34				
Fri																				
			COMP/2004/01/C/01/03 Introduction to Cyber Physical Systems Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr 19-27, 32-33	Computing Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr JC Computer Science B85								COMP/2004/01/C/01/04 Introduction to Cyber Physical Systems Caleb-Solly, Praminda Prof, Felton, Liz Dr, Magassouba, Aly Dr 19-27, 32-33								
			COMP/2004/02/T/01/07 Tutorial Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Exchange C03																	
			COMP/2004/02/T/01/01 Tutorial Algorithms Data Structures and Efficiency Parkes, Andrew Dr 20-27, 32-33 JC Advanced Manufacturing C011																	
			COMP/2014/01/L/02/01 Lecture Distributed Systems Greenhalgh, Chris Prof 19-27, 32, 33 JC Exchange B01	COMP/2010/01/L/02/01 Lecture Software Specification Greensmith, Julie Dr, Luckcruck, M Dr 19-27, 32, 33 JC Exchange C33	COMP/2004/01/L/02/01 Lecture Introduction to Cyber Physical Systems Sailaja, Neelima Dr, Deshpande, Nikhila Dr 19-27, 32 JC Exchange C33	COMP/2012/01/L/02/01 Lecture Languages and Computation Widdershoven, Cas Dr, Alenkirch, Thorsten Prof 19-27, 32-33 JC Business School South Auditorium B52						COMP/2010/01/T/01/01 Tutorial Software Specification Greensmith, Julie Dr, Luckcruck, M Dr 19-27, 32, 33 JC Exchange C33	COMP/2003/01/L/02/01 Lecture Advanced Functional Programming Hutton, Graham Prof 19-27, 32, 33 JC Exchange B01		COMP/2001/01/C/01/01 Artificial Intelligence Methods Ozcan, Ender Prof, Jackson, Warren Dr 19-27, 32-33 JC Computer Science Atrium, JC Computer Science A07, JC Computer Science A32		Computing Ozcan, Ender Prof, Jackson, Warren Dr JC Computer Science A32			

■ University Course Timetabling



Routing

- Travelling Salesman and Vehicle Routing





Supply Chains

- Larger scale routing problems – generally global issues.
- Warehouses and inventory.





Wind Farms

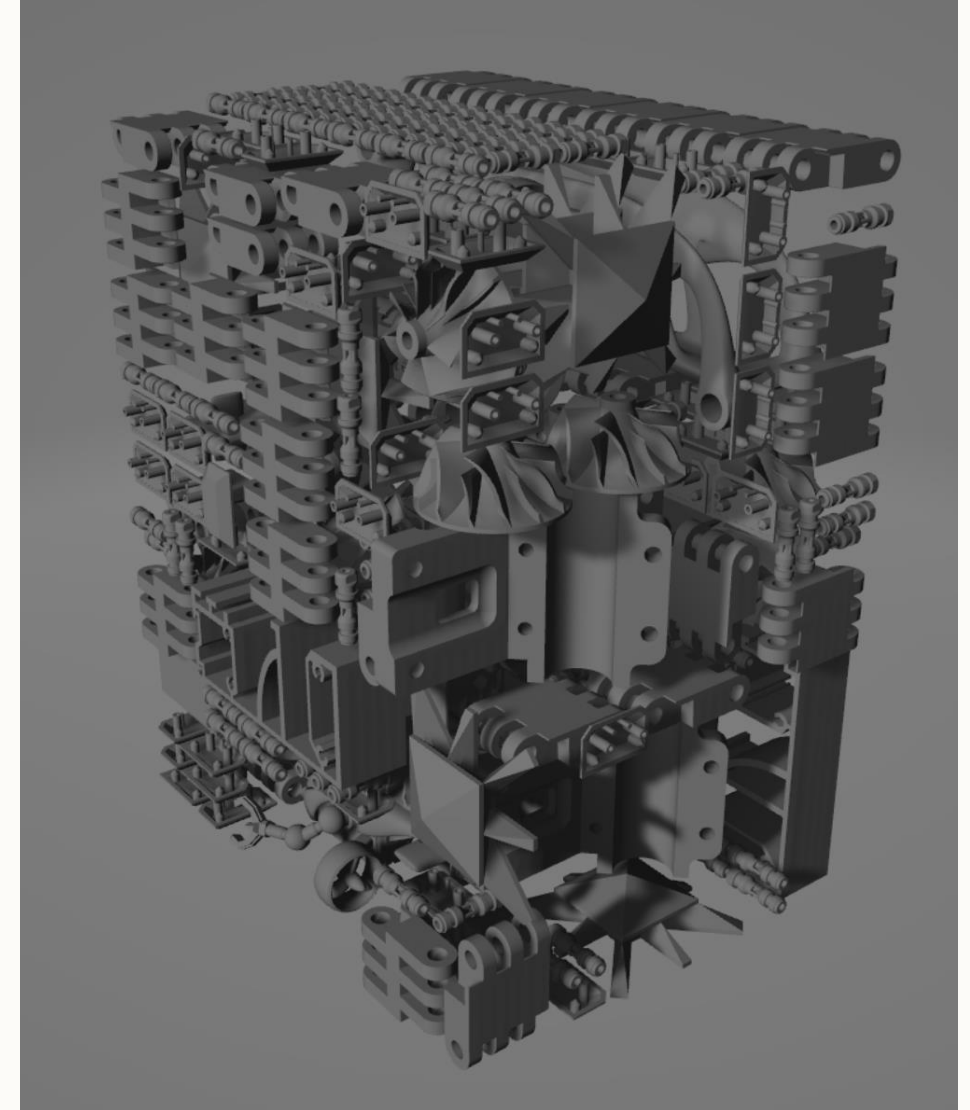
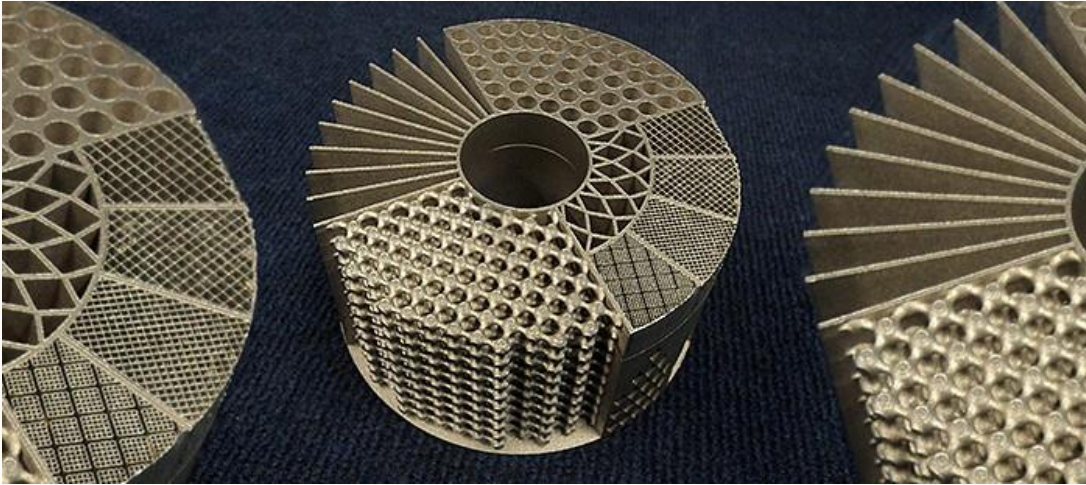
- Wind farm layouts.





Additive Manufacturing

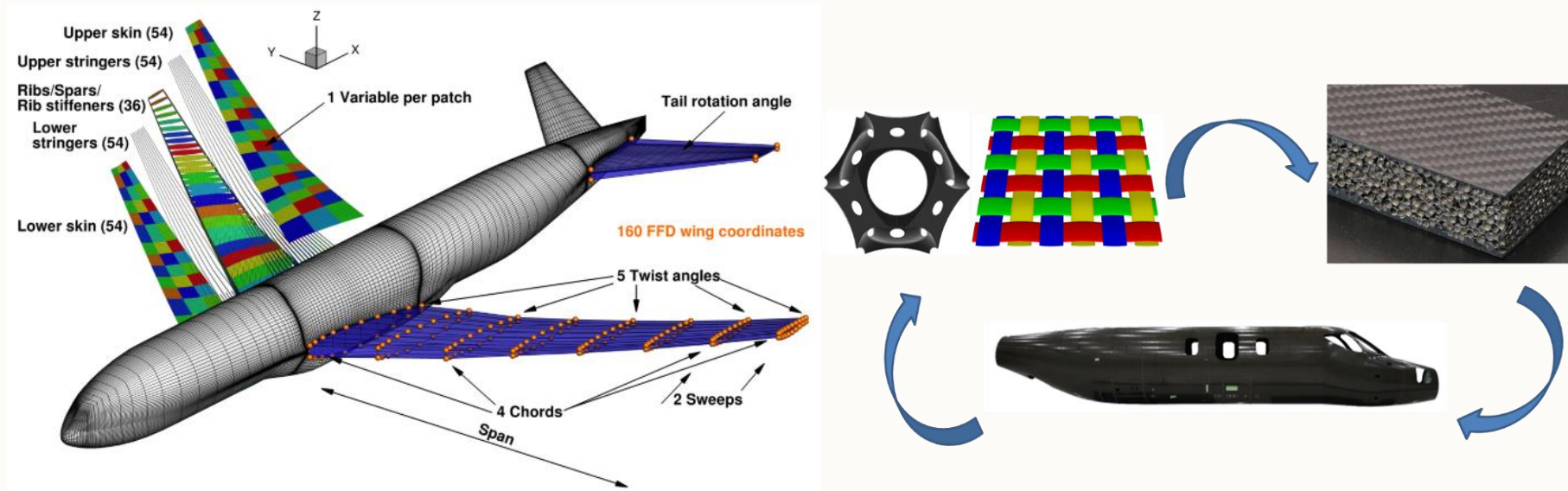
- 3D Structures for material properties.
- Production planning.





Engineering Design

- Noise reduction with composite light-weight materials.



**What did
these have
in common?**





Optimisation/Search Techniques

- These are all real-world problems that can all be “solved” by optimisation/search techniques:
 - Automatically create timetables.
 - Decrease carbon emissions (routing/sequencing).
 - Reduce operational costs (e.g. supply chains).
 - Increase energy production (better windfarm layouts).
 - Improve workflows in additive manufacturing.
 - Optimise structures combining composite and porous layers.
- Note the interchangeable vocabulary – unfortunately quite a lot like this in the field.
- We will study these techniques throughout the course!

Techniques

Genetic Programming,
Grammatical Evolution,
Genetic/Memetic
Algorithms, Heuristics,
Hyper-heuristics, Great
Deluge, Simulated
Annealing



Course Details

What to expect from this module
COMP2001 & COMP2011



Course Details for COMP2011 (10-credits)

- Module Activities:
 - 1 x 2-hour lecture per week.
 - Lecture recordings available on Moodle.
 - Asynchronous activities & formative assessments:
 - Weekly formative quizzes on Moodle.
 - Discussion forums on Moodle for collaborative problem solving.
- Summative Assessment:
 - 1-hour exam (In-person ExamSys) [100%]



Course Details for COMP2001 (20-credits)

- Module Activities:
 - 1 x 2-hour lecture per week.
 - Lecture recordings available on Moodle.
 - 1 x 2-hour computing session – **first lab is this week.**
 - Asynchronous activities & formative assessments:
 - Weekly formative quizzes on Moodle.
 - Discussion forums on Moodle for collaborative problem solving.
- Summative Assessment:
 - Coursework [50%]
 - In-lab exam on exercises (20%)
 - In-lab practical exam on project (30%)
 - 1-hour exam (In-person ExamSys) [50%]



Examinable Material

- Anything said, presented, written on slides, in demos, from formative assessments, and on discussion forums (initiated by the lecturers) is examinable!
- In addition, 20-credit students should expect anything from the lab materials and exercises to be examinable for the coursework.



Main Educational Aims

- Students will have a sound understanding of a wide range of fundamental concepts and techniques of Artificial Intelligence (used for intelligent decision support) with a focus on modern heuristic search/optimisation techniques.
- Students will understand the methods and techniques that are available as an aid in automated decision making/optimisation.
- Students will be acquainted with a number of applications and will understand how software tools are designed to solve them.



Topics COMP2001 & COMP2011

1. Introduction; Heuristic Search/Optimisation; Search Paradigms
2. Components of Search Methodologies; Hill Climbing
3. Metaheuristics: Single Point-based Search
4. Move Acceptance in Metaheuristics and Parameter Setting Issues
5. Evolutionary Algorithms I
6. Evolutionary Algorithms II
7. Hyper-heuristics I
8. Hyper-heuristics II
9. Fuzzy Systems & Symbolic AI [Advanced Topics]
10. Large Language Models & Simulation [Advanced Topics]
11. Revision



Computing Activities & Coursework (COMP2001)

1. Introduction; Heuristic Search/Optimisation; Search Paradigms
2. Components of Search Methodologies; Hill Climbing
3. Metaheuristics: Single Point-based Search
4. Move Acceptance in Metaheuristics and Parameter Setting Issues
5. Evolutionary Algorithms I
6. Evolutionary Algorithms II
8. Hyper-heuristics I & Project Support
9. Hyper-heuristics II & Project Support

Provisionally in-lab exam in week 7, and in-lab practical project exam after Easter break.



Questions and Contacting the Lecturers

- Ask during lectures.
- Look at the Moodle forums. Is your question already there? Can you ask this via Moodle?
- Personal queries – please contact us both directly via email:
 - Ender.Ozcan@nottingham.ac.uk
 - Warren.Jackson2@nottingham.ac.uk
- COMP2001:
 - Support and feedback opportunities will be available in labs.
 - Ask any of us or one of the teaching support team.



Part 1

Preliminaries – Basics and Terminology



Definitions - What is Decision Support?

- A term used in a variety of contexts in decision making.
- Decision Making relates to the process of making a choice between alternatives.
- Multidisciplinary area of research interest to academics and practitioners:
 - Artificial Intelligence
 - Operations Research
 - Decision Theory
 - Decision Analysis
 - Statistics
 - ...and more.



Definitions - What are Systems?

- A **system** can be defined as a number of related elements which together perform an activity, function, or task.
- Two classes based on the dependence on the environment:
 - Closed systems are independent.
 - Open systems are dependent; they interact with the environment.
- How to evaluate systems?
 - **System effectiveness** – the degree to which goals are achieved.
 - **System efficiency** – a measure of the use of inputs/resources to achieve an output.



Solving Problems by Searching

- We can solve problems by:
 - Systematically searching from paths to goals, e.g. tree/graph search, or
 - Searching for solutions (the process of optimisation).



Search for paths to goals

- Involves finding a set of actions that will move from a given initial state towards a goal state.
- Central to many AI problems such as game playing and path finding.
- Seen examples of these in COMP1008:
 - Breadth First Search
 - Depth First Search
 - Uniform Cost Search
 - A*



Search for Solutions

- More general than searching for paths to goals.
- Involves efficiently finding a solution to a problem in a large search space.
- Subsumes searching through paths to goals (a path through a search tree can be encoded as a candidate solution in an optimisation process).
- At a high level, the process of optimisation is to find a better (high quality) solution.



Solving a Single-objective Optimisation Problem 1

- Such problems can be formulated as a mathematical optimisation problem.
- Process of **optimisation** involves **minimising** or **maximising** a quantity (the objective function) the measure/objective value of a solution.
- Quality of a solution expressed by an objective function:
 - $\min z = f(X)$
 - where X is a vector of variables/decisions $\langle x_1, x_2, \dots, x_n \rangle$.
- Often subject to one or more constraints:
 - $x_1 + x_2 \geq 0$
 - $2x_2 + x_3 = 5$
 - ...



Solving a Single-objective Optimisation Problem 2

- Variables may or may not be bounded,
- ...and may be continuous or discrete.
- Could have an infinitely large search space.
- Systems of equations generally indifferentiable.
- How to find the optimal solution? Need to choose a mathematical or search method to solve the optimisation problem.



Exercise – Ackley Function Demo

- Can you find the optimal values of x_1 and x_2 to the following function?
- $$f(x) = -20 \exp \left(-0.2 \sqrt{\frac{1}{2} \sum_{i=1}^2 x_i^2} \right) - \exp \left(\frac{1}{2} \sum_{i=1}^2 \cos(2\pi x_i) \right) + 20 + \exp(1)$$
- people.cs.nott.ac.uk/pszeo/teaching/javaScript/G52AIM/ackley.html

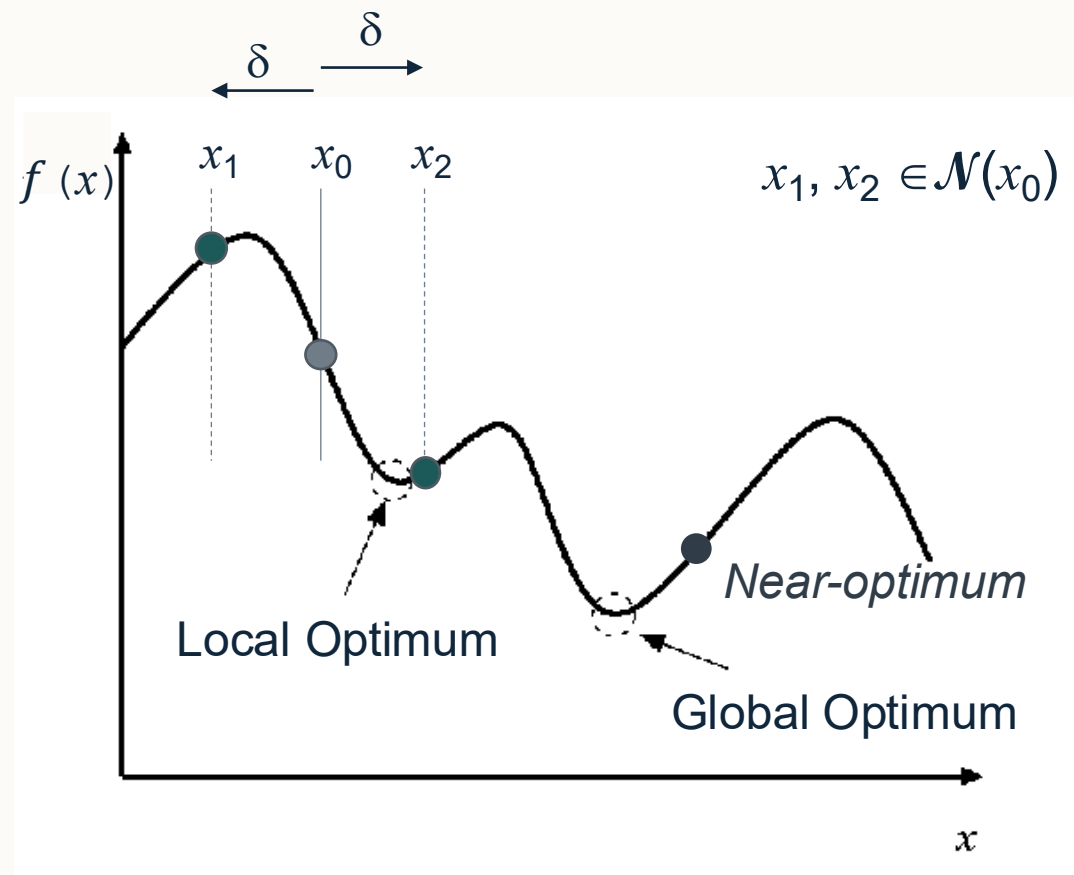
- What strategies did you come up with to search?
- Heuristics





Local vs Global Optimum

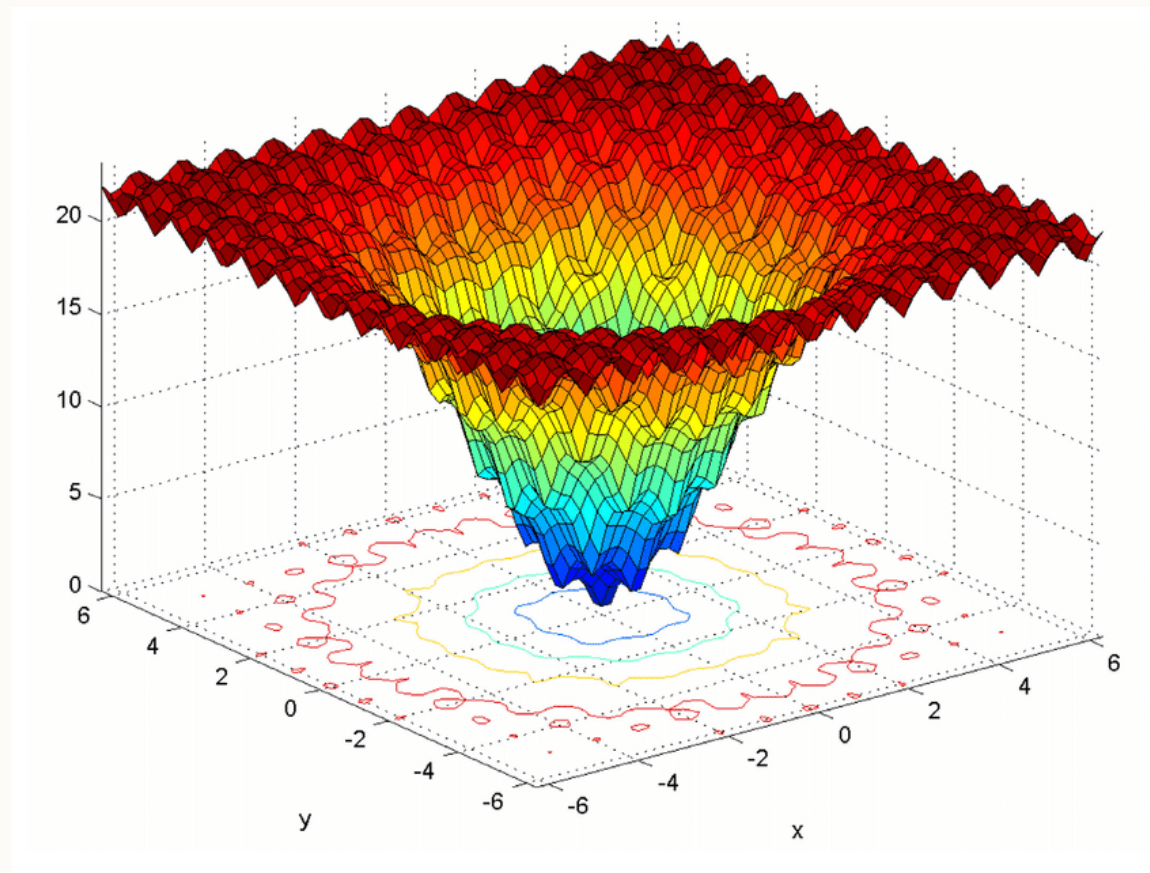
- Global optimum: better than all other solutions.
- Local optimum: better than all solutions in a given neighbourhood, $\mathcal{N}$.
- Example:
 - 1D search space
 - $\mathcal{N}$ defined as $\pm\delta$





Ackley Function Search Landscape

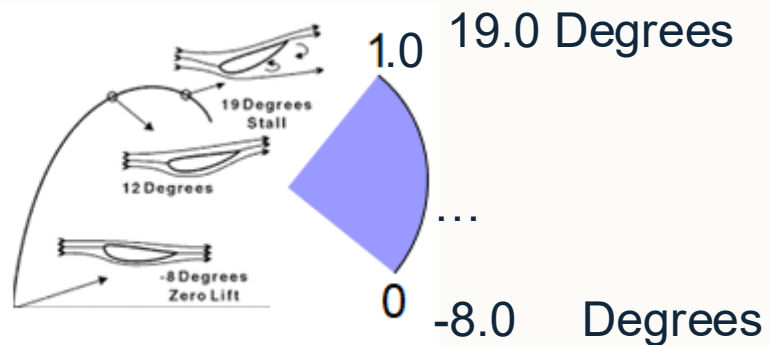
- Search landscape can be difficult for search algorithms to navigate.
- With a local view it is not possible to know where the best solution is.
- Common issue is to get stuck somewhere that is locally optimum.





Continuous vs Discrete Search Spaces 1

- Find the optimum setting for the angle of the wings of a race car to give the best performance.



- Assign tourist groups to busses to produce the optimum number of 70-seater tour busses for tourist groups of size: $\{30, 10, 60, 40, 50, \dots, 35\}$.





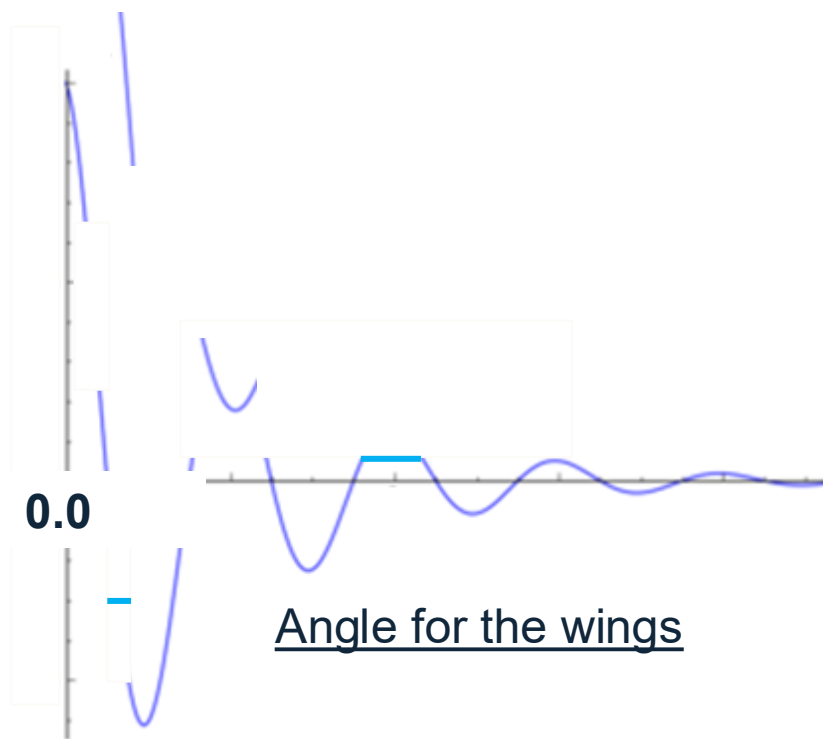
Tourist Bus Problem

- Assign tourist groups to busses produce the optimum number of 70-seater tour busses for tourist groups of size: $\{30, 10, 60, 40, 50, \dots, 35\}$.
- We can easily find **what** the optimum is.
 - $\frac{1}{70} \sum_{i=1}^n x_i$
- But not **where** the optimum is.
 - Which groups go in which busses?



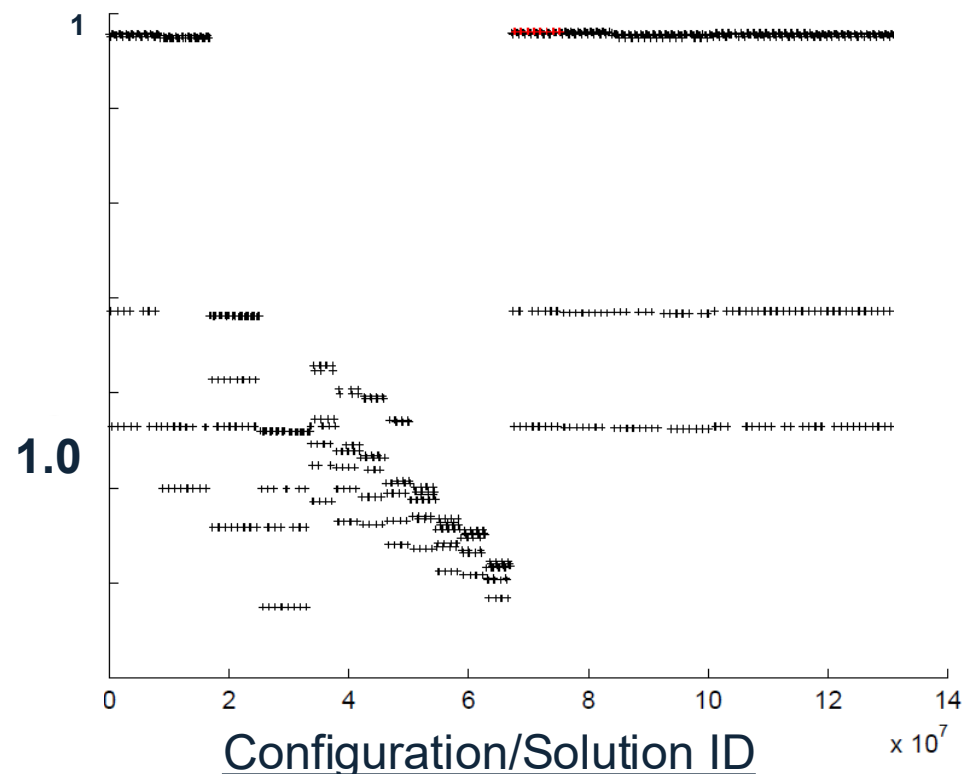
Continuous vs Discrete Search Spaces 2

Performance



Real-valued (parameter)/
Continuous Optimisation

Average bus fullness



Organising tour busses



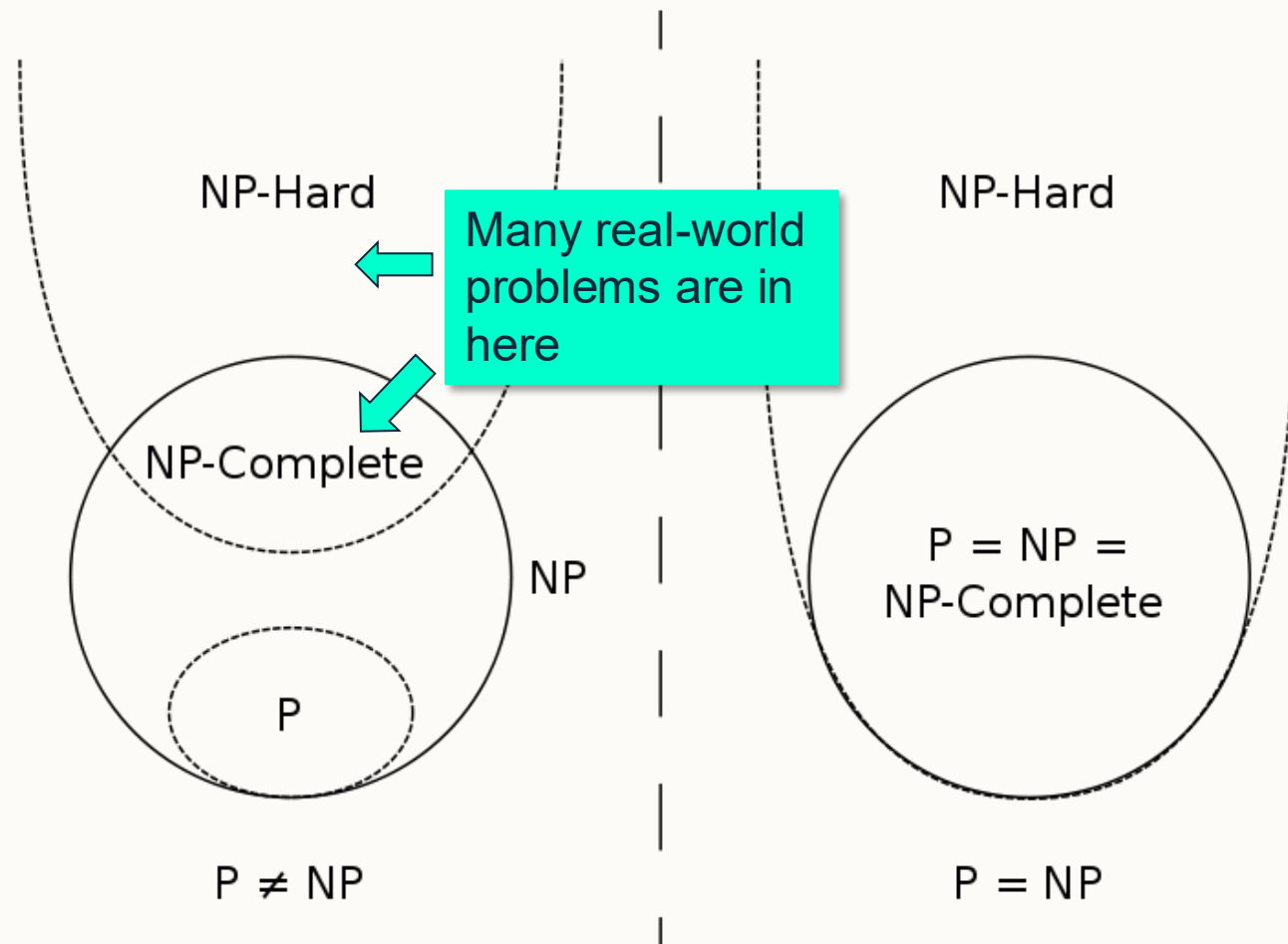
Definitions – Problem and Problem Instance

- A **problem** is the high-level question/optimisation issue to be solved.
- An **instance** of a problem is a concrete expression which represents the input for a decision or optimisation problem.
- Example with tourist bus problem.



Complexity Classes

- COMP2054 & COMP3001
- Most real-world problems lie in NP-Hard and NP-complete problem spaces.
- These are known as combinatorial optimisation problems and are the problems we will focus on in this course.





Combinatorial Optimisation Problems

- Require finding an optimal solution from a finite set of solutions.
- For NP-Hard COPs, this finite set grows exponentially with the instance size.
- Hence becomes intractable to search through all possible solutions for anything other than the smallest of instances.
- Problems we discussed at the beginning are all examples of COPs!



Optimisation/Search Methods

- Broadly classified into two approaches:
 - Exact/exhaustive/systematic methods
 - Inexact/approximate/local search methods (our focus)
- Exact methods
 - Dynamic programming, Branch and Bound, Constraint Satisfaction
 - Systematically traverse the search space.
 - Has optimality guarantees.
- Inexact methods
 - Heuristics/Metaheuristics/Hyper-heuristics
 - Sample the search space through neighbourhood operators.
 - No optimality guarantees.



Search Paradigms

- Single-point (trajectory) vs. multi-point (population).
- Perturbative heuristics operate on complete solutions.
- Constructive heuristics operate on partial solutions.
- **Deterministic vs stochastic.**
- Systematic vs **local search.**
- **Sequential** vs parallel.
- **Single-objective** vs multi-objective.

Further reading - [See <http://www.cs.ubc.ca/~hoos/SLS-Internal/ch1.pdf> pp.23-30]

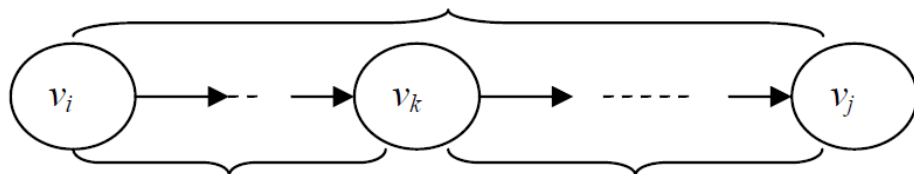


Optimisation Techniques 1

Exact/Exhaustive/Deterministic Systematic Methods

■ Dynamic Programming

A shortest path from v_i to v_j using only vertices in $\{v_1, v_2, \dots, v_{k-1}, v_k\}$



A shortest path from v_i to v_k using only vertices in $\{v_1, v_2, \dots, v_{k-1}\}$

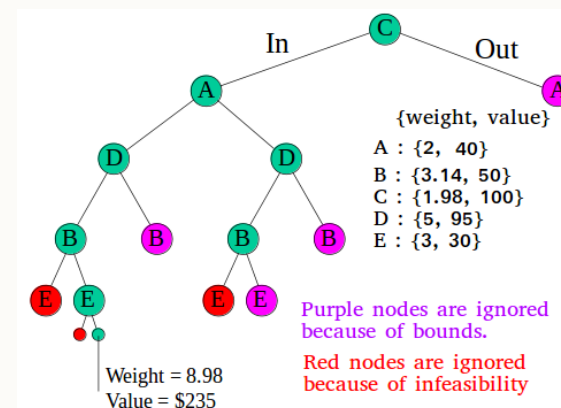
A shortest path from v_k to v_j using only vertices in $\{v_1, v_2, \dots, v_{k-1}\}$

■ Constraint Satisfaction

Limitations:

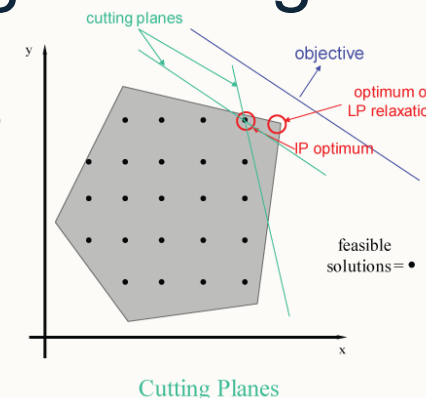
- Only work if the problem is structured – in many cases for small problem instances
- Quite often used to solve sub-problems

■ Branch & Bound



■ Math programming

- ❖ ILP
- ❖ MILP





Example 1 – Bin Packing Problem Instance

- The underlying problem in CS for the tourist bus problem.
- Pack set of items with varying sizes into a number of fixed capacity bins while minimising the total number of bins required.
- $V = 254$
- $n = 33$
- Items: 85, 442, 10, 10, 10, 10, 10, 10, 252, 252, 252, 252, 252, 252, 252, 9, 9, 127, 127, 127, 127, 127, 106, 106, 106, 106, 12, 12, 12, 84, 46, 37



Optimisation Techniques 1 – Bin Packing

Mathematical Programming

■ COMP4038/4041

Model

$$\text{minimize } B = \sum_{i=1}^n y_i$$



subject to $B \geq 1$,

$$\sum_{j=1}^n a_j x_{ij} \leq V y_i, \forall i \in \{1, \dots, n\}$$

$$\sum_{i=1}^n x_{ij} = 1, \quad \forall j \in \{1, \dots, n\}$$

$$y_i \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\}$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, \dots, n\} \forall j \in \{1, \dots, n\}$$

where $y_i = 1$ if bin i is used and $x_{ij} = 1$ if item j is put into bin i .^[5]

```
{string} bins = ...;
{string} items = ...;
float itemSize[bins] = ...;
float binCapacity[bins] = ...;
range r = 0..1;
dvar int X[items][bins] in r;

minimize sum(u in bins) pow(((sum(a in items) itemSize[a]*X[a][u])), 2);

subject to {
    /* Item in only one bin constraint */
    forall(i in items)
        sum(b in bins)
            X[i, b] == 1;

    /* Capacity Constraint */
    forall(b in bins)
        sum(i in items) X[i][b]*itemSize[i] <= binCapacity[b];
};

execute DISPLAY {
    writeln("Mapping:");
    for (var i in items) {
        for (var b in bins) {
            if (X[i][b] == 1) {
                writeln("Item " + i + " is in bin " + b);
            }
        }
    }
}
```

```
itemSize = #[ 1: 85, 2: 442, 3: 10, 4: 10, ..., 32: 46, 33: 37 ]#;
binCapacity = #[ 1: 524, 2: 524, 3: 524, ..., 33: 524 ]#;
```



Example 1 – Optimal Bin Packing Performance Comparison with Different B&B Algorithms

- The motivation for inexact methods.
- Solution time increases substantially as the problem size increases
- 25 days for an instance $n = 60$

N	Optimal	L2 bound	% Optimal		Martello + Toth		Bin Completion		Ratio
			FFD	BFD	Nodes	Time	Nodes	Time	Time
5	3.215	3.208	100.000%	100.000%	0.000	7	.013	6	1.17
10	5.966	5.937	99.515%	99.541%	.034	15	.158	13	1.15
15	8.659	8.609	99.004%	99.051%	.120	25	.440	19	1.32
20	11.321	11.252	98.570%	98.626%	.304	37	.869	27	1.37
25	13.966	13.878	98.157%	98.227%	.741	55	1.500	36	1.53
30	16.593	16.489	97.790%	97.867%	2.146	87	2.501	44	1.98
35	19.212	19.092	97.478%	97.561%	7.456	185	4.349	55	3.36
40	21.823	21.689	97.153%	97.241%	39.837	927	8.576	73	12.70
45	24.427	24.278	96.848%	96.946%	272.418	6821	20.183	103	66.22
50	27.026	26.864	96.553%	96.653%	852.956	20799	57.678	189	110.05
55	29.620	29.445	96.304%	96.414%	6963.377	200998	210.520	609	330.05
60	32.210	32.023	96.036%	96.184%	58359.543	2153256	765.398	2059	1045.78
65	34.796	34.598	95.780%	95.893%			11758.522	28216	
70	37.378	37.167	95.556%	95.684%			16228.245	41560	
75	39.957	39.736	95.322%	95.447%			90200.736	194851	
80	42.534	42.302	95.112%	95.248%			188121.626	408580	
85	45.108	44.866	94.854%	94.985%			206777.680	412576	
90	47.680	47.428	94.694%	94.832%			1111759.333	2522993	

See <http://www.aaai.org/Papers/AAAI/2002/AAAI02-110.pdf>



Optimisation Techniques 2

- Inexact/approximate methods can solve* COPs effectively and efficiently.
- Are the focus of this course.

- *Caveats discussed later.



Part 2

Heuristic Search and Optimisation



Heuristic Search Methods

- A **heuristic** is a rule of thumb method derived from human intuition.
- Heuristics are:
 - Problem dependent search methods which seek good solutions in a reasonable computational budget.
 - Good for solving ill-structured problems, or well-structured problems that are complex.
 - Not able to guarantee optimality (unless you can mathematically prove otherwise).



Example 1 – Bin Packing Problem Instance

- $V = 524$
- $n = 33$
- Items: 85, 442, 10, 10, 10, 10, 10, 10, 252, 252, 252, 252, 252, 252, 252, 9, 9, 127, 127, 127, 127, 127, 106, 106, 106, 106, 12, 12, 12, 84, 46, 37
- How can we solve this problem using a heuristic/inexact method?

Peer discussion – what heuristics can you come up with?



Example 1 – A Heuristic Solution

- **Largest item first fit** sorts the items by size in descending order, then places each next item into the first open bin with capacity.

Bin	Items						
a_1	442	252	127	106	37	10	10
a_2	252	252	127	106	37	10	9
a_3	252	252	127	85	12	10	9
a_4	252	127	106	84	12	10	
a_5	252	127	106	46	12	10	

- $V=524$
- $n=33$
- Item sizes (33 items to pack): 85, 442, 10, 10, 10, 10, 10, 10, 10, 252, 252, 252, 252, 252, 252, 252, 9, 9, 127, 127, 127, 127, 127, 106, 106, 106, 106, 12, 12, 12, 84, 46, 37



Largest item first fit rule/heuristic



	Bin1	Bin2	Bin3	Bin4	Bin5	Bin6	Bin7
442	442						
252		252					
252		252					
252			252				
252			252				
252				252			
252				252			
252					252		
127					127		
127					127		
127						127	
127						127	
127						127	
106						106	
106							106
106							106
106							106
85							85
84							84
46 – removed							
37						37	
37							37
12	12						
12	12						
12	12						
10		10					
10		10					
10			10				
10			10				
10				10			
10				10			
9					9		
9					9		
	524	524	524	524	524	524	524

Instance#1

Bin1	Bin2	Bin3	Bin4	Bin5	Bin6	Bin7	Bin8
442							
	252						
	252						
		252					
		252					
			252				
			252				
				252			
				127			
				127			
					127		
					127		
					106		
						106	
						106	
						106	
						85	
						84	
37							
37							
	12						
		12					
			12				
				10			
					10		
					10		
						10	
						10	
						9	
							9
516	516	516	516	516	517	516	9

Instance#2



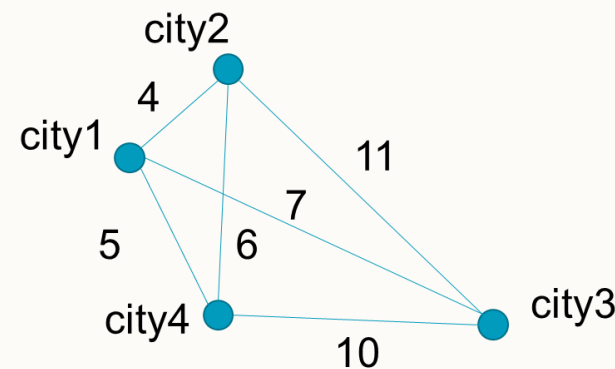
CASE STUDY

A comparison of perturbative and constructive stochastic local search algorithms using a case study for the Travelling Salesman Problem



Travelling Salesman Problem (TSP)

- Fundamental CS problem which underpins many real-world problems, including planning, logistics, vehicle routing, telecommunication, manufacturing of microchips, genome sequencing, scheduling, cutting&packing, and more...
- Informally, “Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city?”
- Minimise $z = f(X)$ where X is a vector of variables $\langle x_1, x_2, \dots, x_n \rangle$
- Instance: city_1, city_2, city_3, city_4 where $n = 4$.
- Example solutions include (path representation):
 - $\langle \text{city_3}, \text{city_1}, \text{city_2}, \text{city_4} \rangle$
 - $\langle \text{city_2}, \text{city_1}, \text{city_4}, \text{city_3} \rangle$
- Exercises: Permutation of 5 cities [\[E1\]](#), Permutation of 20 cities [\[E2\]](#), Exhaustive Search: Solving TSP with 5 cities [\[E3\]](#)





The Need for Inexact Methods

- Travelling salesman problem with N cities

- $N=4$, 24

- $N=5$, 120

- $N=7$, 5 040

- $N=10$, 3 628 800

- $N=81$, 5.797×10^{120}



- Number of configurations to search from is $N!$
- Number of particles in the universe is in between $10^{72} - 10^{87}$
- El Capitan (2024): $\sim 1,742$ PetaFLOPS (one thousand million (10^{15}) floating-point operations per second) – $\sim 1.06 \times 10^{95}$ years (from [TOP500](#) project).

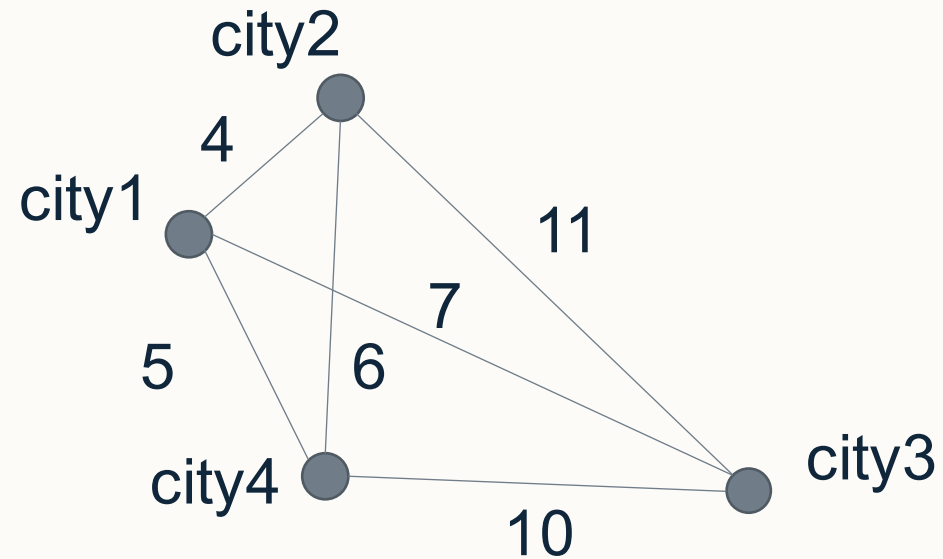


Some Heuristics for TSP

- Nearest neighbour (NN) algorithm.
 - Constructive
 - Stochastic
 - Systematic
- Pairwise Exchange (2-opt)
 - Perturbative
 - Stochastic
 - Local Search
- Example operation on following slides.



Nearest Neighbour (NN) Algorithm





Nearest Neighbour (NN) Algorithm

- Step 1: choose a starting city randomly.
- Solution: <city2>

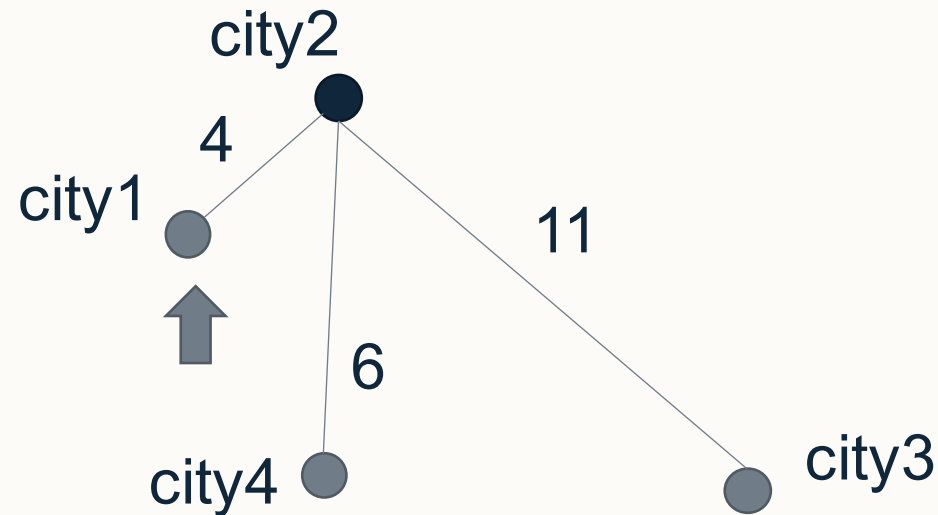
city2





Nearest Neighbour (NN) Algorithm

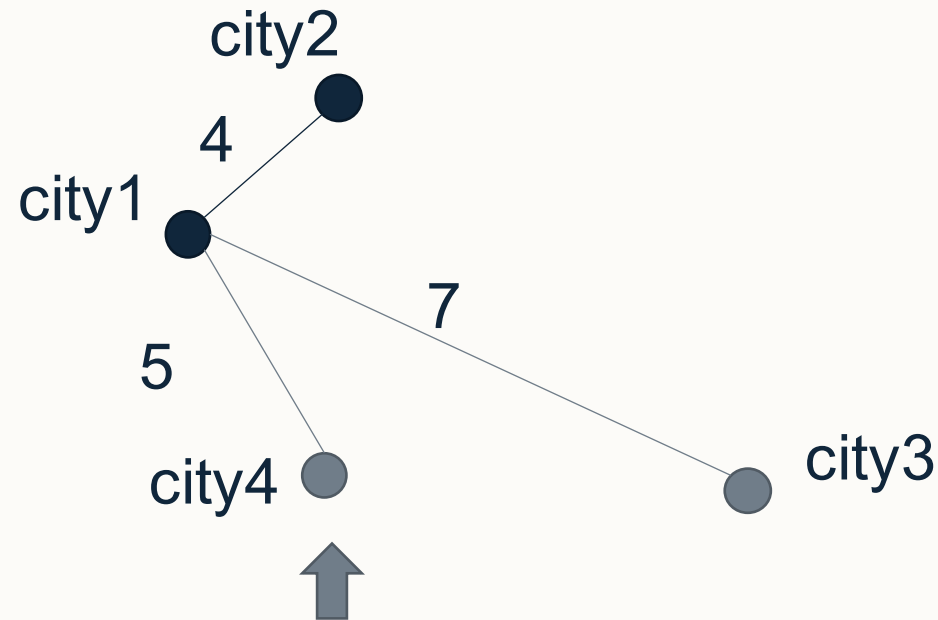
- Step 2: choose the nearest unvisited city as the next city and repeat until all cities are chosen.
- Solution: $\langle \text{city2}, \text{city1} \rangle : 4$





Nearest Neighbour (NN) Algorithm

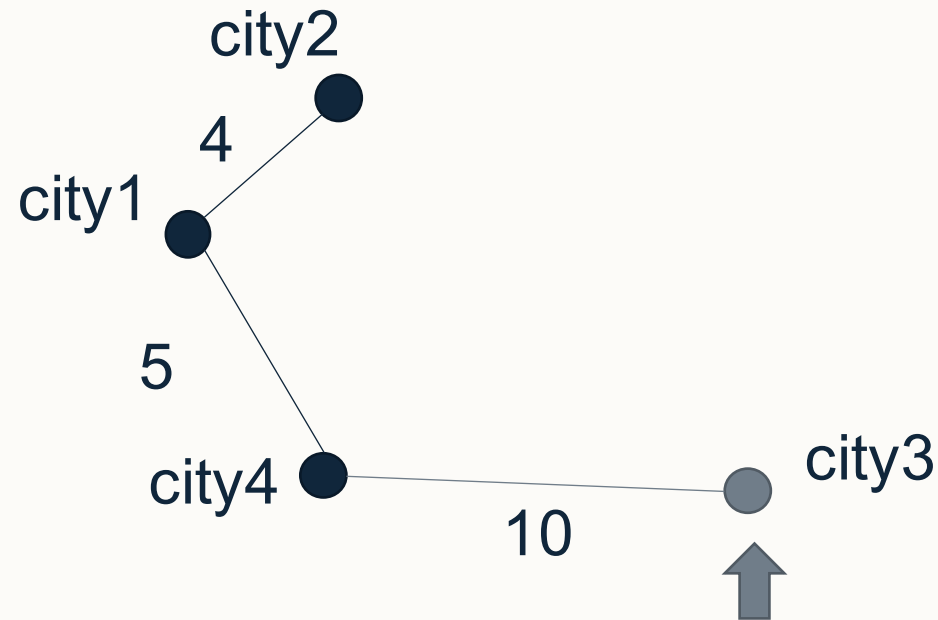
- Step 3: choose the nearest unvisited city as the next city and repeat until all cities are chosen.
- Solution: $\langle \text{city2}, \text{city1}, \text{city4} \rangle : 4+5=9$





Nearest Neighbour (NN) Algorithm

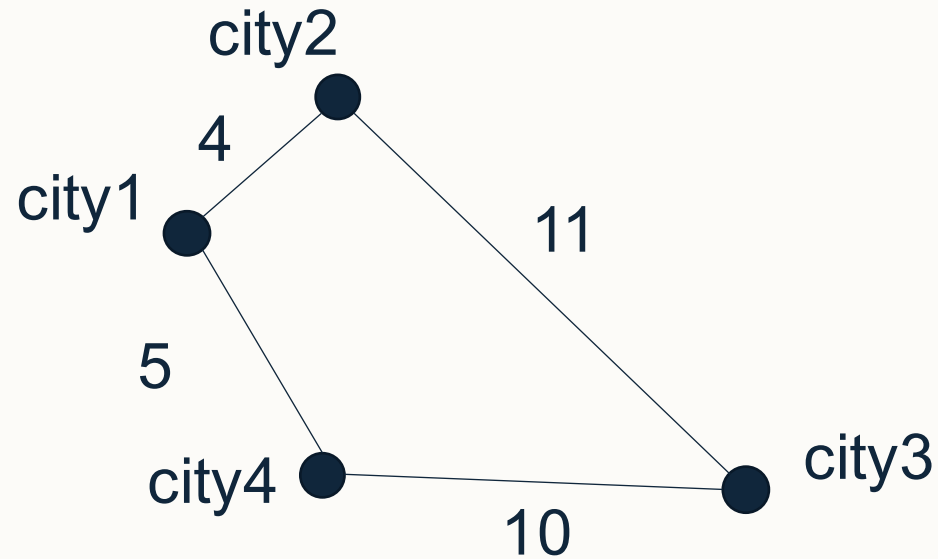
- Step 4: choose the nearest unvisited city as the next city and repeat until all cities are chosen.
- Solution: $\langle \text{city2}, \text{city1}, \text{city4}, \text{city3} \rangle : 9+10=19$





Nearest Neighbour (NN) Algorithm

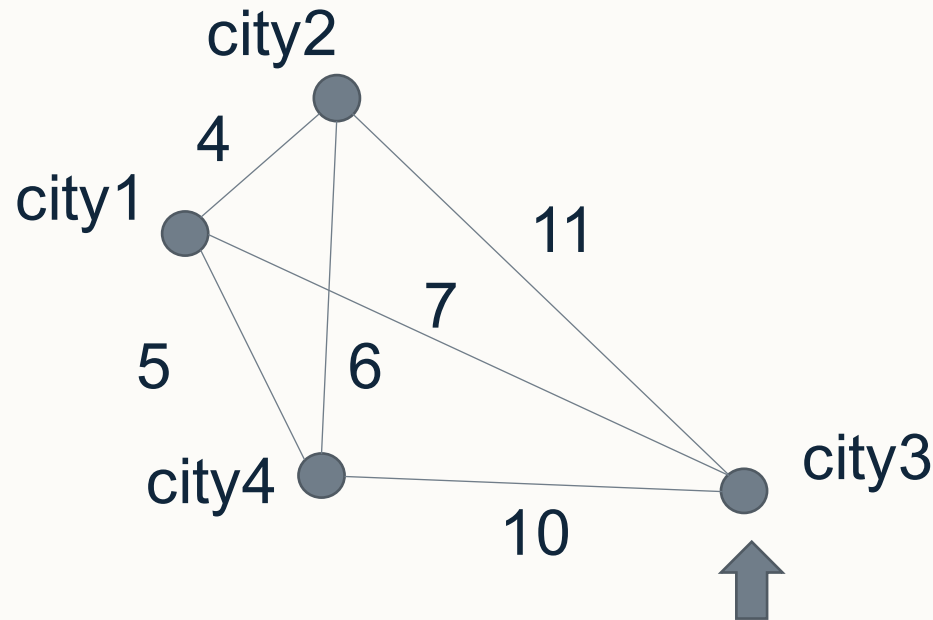
- Step 5: All cities are now chosen so the algorithm terminates.
- Solution: $\langle \text{city2}, \text{city1}, \text{city4}, \text{city3} \rangle : 19+11=30$





Nearest Neighbour (NN) Algorithm - Notice

- What happens if a different starting city is chosen?



- Solution: $\langle \text{city3}, \text{city1}, \text{city2}, \text{city4} \rangle : 27$



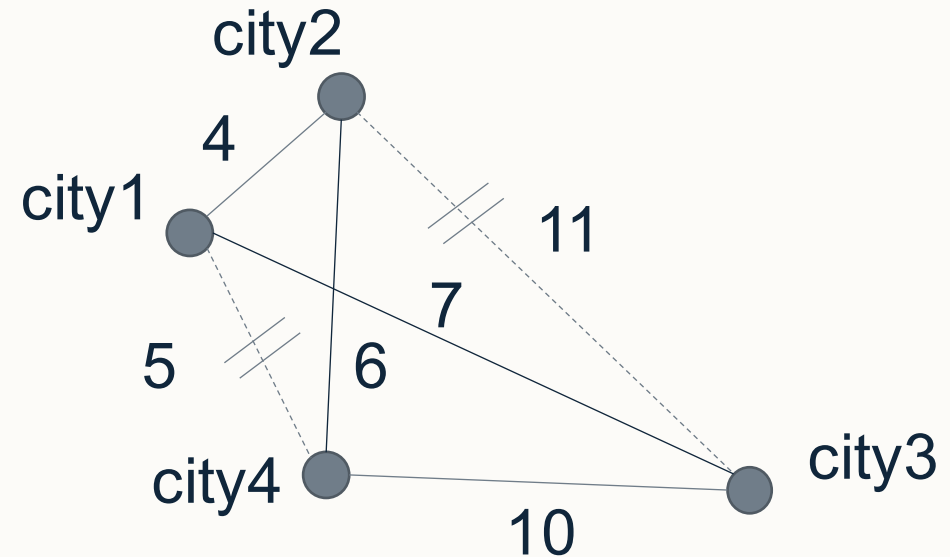
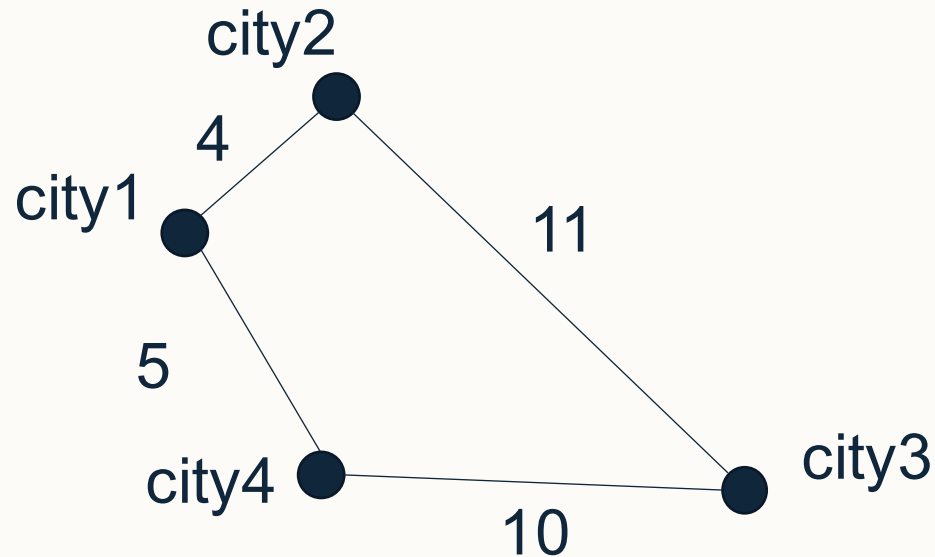
A Constructive Stochastic Local Search Algorithm for TSP

- Step 1: Choose a random city.
- Step 2: Apply NN to construct a complete solution.
- Step 3: Compare the new solution to the best solution found so far and update if we improved.
- Step 4: **Repeat** until some maximum number of iterations is reached by going back to Step 1.
- Step 5: **Return** the best solution found.



Pairwise Exchange (2-Opt)

- Solution: $\langle \text{city2}, \text{city1}, \text{city4}, \text{city3} \rangle : 30$
- Remove two edges and replace them with two different edges, reconnecting into a new and complete tour.
- Corresponds to swapping two cities in the path representation.
- New solution: $\langle \text{city1}, \text{city2}, \text{city4}, \text{city3} \rangle : 27$





A Perturbative Stochastic Local Search Algorithm for TSP

- Step 1: Create a random current solution.
- Step 2: Apply pairwise exchange to form a new solution.
- Step 3: Compare the new solution to the current solution replacing the current solution with the new solution if an improvement is made.
- Step 4: **Repeat** until some maximum number of iterations is reached by going back to Step 2.
- Step 5: **Return** the current solution.



Optional Further Reading - TSP

- Other Heuristics for solving TSP
 - Christofides' algorithm (1976)
 - Match Twice and Stitch [[doi:10.1016/j.orl.2004.04.001](https://doi.org/10.1016/j.orl.2004.04.001)]
 - Lin–Kernighan [[doi:10.1016/S0377-2217\(99\)00284-2](https://doi.org/10.1016/S0377-2217(99)00284-2)]
- Exact algorithm – Concorde:
<http://www.math.uwaterloo.ca/tsp/concorde/index.html>
- TSP benchmark data: [[University of Waterloo](http://www.math.uwaterloo.ca/tsp/concorde/index.html)]



Caveats of Heuristic Search

- There are no optimality guarantees (contrast to exact methods).
- Usually require designing new heuristics for solving other problems.
- Often parameterised and very sensitive to their settings.
- May result in poor quality solutions when:
 - Used for different problem instances.
 - Applied a second time (stochastic methods).



Part 3

Pseudo-random numbers



Deterministic vs Stochastic Heuristic Search

- Deterministic heuristic search algorithms provide the same solution when run on the given problem instance regardless of how many times.
- Stochastic algorithms contain a random component and may return a different solution at each time they are run on the same instance
 - Multiple trials/runs should be performed for the experiments/simulations
 - Being able to repeat/replicate those multiple trials/runs in the experiments/simulations is crucial in science, and
 - This also enables average performance comparison of different stochastic heuristic search algorithms applying statistical tests



Pseudo-random numbers

- Generate a sequence of numbers in a deterministic way but appear to be random.
- Most (not all) computers and programming languages support seeding of pseudo-random number generators.
 - E.g. `rand()` and `srand()` in C,
 - `new Random(int seed)` in Java,
 - ...
- Why is seeding useful for experimentation?
 - Explore sample of algorithm performance.
 - Guaranteed repeatability of results.



Issues with pseudo-random number generators

- The period of a pseudo-random number generator refers to the length of a sequence before being repeated – what if this is too short?
- Lack of uniformity of distribution of values.
- Correlation of successive values.
- Generally not an issue with modern methods.
 - E.g. Java's Random has a period of 2^{48}
- Cryptographically secure random number generators exist – why would these be a bad idea?



An Example – Middle Square Method

- John Von Neumann (1949).
- To generate a sequence of n-digit pseudorandom numbers.
- Starts with an n-digit seed, squares the current number and takes the middle n-digits.
- **Example:**
 - Given the seed 2156.
 - $2156^2 = 04648336$
 - $6483^2 = 42029289$
- However all sequences eventually repeat themselves.



Case Study

Pseudo-random numbers and experimentation



Exercise 1

```
for (int trial=0; trial<5; trial++)  
// Assume this represents 5 trials/runs of an experiment  
{  
    long seed = 123456789;  
    Random generator = new Random(seed);  
    double num = generator.nextDouble() ; // rand. value between 0.0 and 1.0  
    System.out.print(num);  
    System.out.print(' ');  
}
```

people.cs.nott.ac.uk/pszeo/teaching/javaScript/G52AIM/I1_4_random1.html



Exercise 2

```
for (int trial=0; trial<5; trial++)  
// Assume this represents 5 trials/runs of an experiment  
{  
    long seed = System.currentTimeMillis();  
    Random generator = new Random(seed);  
    double num = generator.nextDouble() ; // rand. value between 0.0 and 1.0  
    System.out.print(num);  
    System.out.print(' ');  
}
```

people.cs.nott.ac.uk/pszeo/teaching/javaScript/G52AIM/I1_4_random2.html



Exercise 3

```
// Assume this represents an experiment
long seed = 123456789;
Random generator = new Random(seed);
for (int trial=0; trial<5; trial++) // 5 trials/runs
{
    double num = generator.nextDouble() ; // rand. value between 0.0 and 1.0
    System.out.print(num);
    System.out.print(' ');
}
```

people.cs.nott.ac.uk/pszeo/teaching/javaScript/G52AIM/I1_4_random3.html



Exercise 4

```
// Assume this represents an experiment
long[] seed = {123456, 789000, 323241, 5523525, 2432342};
for (int trial=0; trial<5; trial++) // 5 trials/runs
{
    Random generator = new Random(seed[i]);
    double num = generator.nextDouble() // rand. value in [0.0,1.0)
    System.out.print(num);
    System.out.print(' ');
}
```

people.cs.nott.ac.uk/pszeo/teaching/javaScript/G52AIM/I1_4_random4.html



Concluding remarks

- Module activities:
 - Formative self-assessment quiz on Moodle.
 - Computing Session (COMP2001): Friday 30th January CS-A32 16:00 – Support for Topic 1: Preliminaries and Pseudo-random Number Generators.
 - Exercises and resources available after the lecture.
 - Next lecture: 3rd February 16:00 – Components of Heuristic Search.



University of
Nottingham

UK | CHINA | MALAYSIA

Thank you

Any questions?